

RELATÓRIO TÉCNICO — PARTE TEÓRICA E DECISÕES ARQUITETURAIS

Loja Veloz — Cloud DevOps & Microservices Modernization

Autor: William Kerpen RA 2903

Curso: Cloud DevOps Orchestrating Containers and Micro Services

Instituição: UniFECAF

Professores: Felipe Becker Nunes, Fernando Leonid e Vitor Santana de Jesus

1. Introdução

O crescimento acelerado da Loja Veloz nos últimos meses trouxe desafios significativos para o ambiente de produção. A empresa passou a enfrentar indisponibilidades durante deploys, dificuldade para escalar em períodos de pico e baixa rastreabilidade de falhas entre serviços. Como Cloud DevOps Engineer responsável pelo projeto, meu objetivo foi propor uma arquitetura moderna, baseada em microserviços, containerização, orquestração e práticas cloud-native, capaz de melhorar a confiabilidade, reduzir riscos e acelerar o ciclo de entrega.

Este relatório apresenta a fundamentação teórica que orientou o desenvolvimento da solução, além das decisões técnicas adotadas no projeto prático. A abordagem segue as orientações oficiais do trabalho, integrando conceitos de microserviços, DevOps, Docker, Kubernetes, CI/CD, observabilidade e estratégias de deploy e escalabilidade.

2. Arquitetura de Microserviços e o Papel do DevOps em Ambientes Cloud-Native

A arquitetura de microserviços é um estilo arquitetural que organiza uma aplicação como um conjunto de serviços independentes, cada um responsável por uma funcionalidade específica. Diferente de aplicações monolíticas, onde tudo está acoplado em um único bloco, os microserviços permitem que cada módulo tenha seu próprio ciclo de vida, podendo ser desenvolvido, testado, implantado e escalado de forma isolada.

No contexto da Loja Veloz, mesmo que a implementação prática tenha sido realizada em uma única API modular, a estrutura interna foi organizada seguindo princípios de microserviços: módulos independentes para produtos, categorias, usuários, pedidos, pagamentos, carrinho, relatórios, suporte, devoluções, logs e healthcheck. Essa modularização facilita a evolução futura para uma arquitetura totalmente distribuída.

O papel do DevOps nesse cenário é fundamental. DevOps integra desenvolvimento e operações para garantir que o software seja entregue de forma rápida, segura e confiável. Em ambientes cloud-native, DevOps se apoia em automação, pipelines CI/CD, containerização, orquestração e observabilidade. A cultura DevOps busca reduzir o tempo entre desenvolvimento

e produção, padronizar ambientes, automatizar deploys e garantir que falhas possam ser detectadas e corrigidas rapidamente.

No projeto da Loja Veloz, a adoção de práticas DevOps foi essencial para padronizar o ambiente local, automatizar o build e o deploy, garantir versionamento adequado das imagens e permitir que a aplicação evolua com segurança.

3. Containerização: Docker, Docker Compose e Kubernetes

A containerização é o processo de empacotar uma aplicação com todas as suas dependências em um ambiente isolado e portátil chamado container. Containers são leves, rápidos, reproduzíveis e ideais para ambientes distribuídos.

3.1 Docker

Docker é a plataforma responsável por criar imagens, rodar containers e gerenciar redes e volumes. No ambiente local da Loja Veloz, Docker foi utilizado para padronizar o desenvolvimento, garantindo que todos os serviços subissem de forma consistente.

O Dockerfile do projeto segue boas práticas:

- multi-stage build para reduzir tamanho da imagem
- uso de usuário não-root
- dependências mínimas
- camadas otimizadas

O Docker Compose foi utilizado para orquestrar o ambiente local, subindo API e PostgreSQL com um único comando, além de definir redes, volumes e variáveis de ambiente.

3.2 Kubernetes

Kubernetes é um orquestrador de containers que gerencia deploys, escalabilidade, balanceamento de carga, reinício automático e políticas de segurança. Ele é essencial para ambientes de produção que exigem alta disponibilidade.

No projeto, mesmo sem utilizar Kubernetes na prática devido a limitações financeiras, toda a arquitetura foi construída com base nos conceitos de:

- | | |
|---------------|-----------------------------------|
| • Deployments | • Secrets |
| • Services | • Horizontal Pod Autoscaler (HPA) |
| • Ingress | • Jobs |
| • StatefulSet | • Namespace isolation |
| • ConfigMaps | |

A estrutura de manifests foi incluída no repositório, e reproduzida com o Minikube, um cluster Kubernetes local, usado para testar deploys, ingress, serviços e pods no seu próprio computador demonstrando como a aplicação subiria em um cluster real.

3.3 Quando usar Docker e quando usar Kubernetes

Docker é ideal para:

- desenvolvimento local
- ambientes pequenos
- aplicações simples
- pipelines CI/CD
- deploy em VMs

Kubernetes é ideal para:

- alta disponibilidade
- múltiplas réplicas
- escalabilidade automática
- ambientes distribuídos
- governança avançada

No caso da Loja Veloz, Kubernetes é o caminho natural para evoluções futuras.

4. Orquestração de Containers

Orquestração é o processo de gerenciar containers em produção, garantindo que eles funcionem de forma coordenada. Isso inclui:

- escalabilidade
- resiliência
- atualizações sem downtime
- monitoramento
- reinício automático
- balanceamento de carga

Kubernetes é a principal ferramenta de orquestração, mas Docker Compose cumpre esse papel no ambiente local.

No projeto, a orquestração foi planejada para:

- subir API e banco com um único comando
- garantir isolamento entre serviços

- permitir migração futura para Kubernetes
- facilitar testes e desenvolvimento

5. CI/CD em Ambientes Distribuídos

CI/CD é o conjunto de práticas que automatiza build, testes, validações, publicação de imagens e deploy. Em ambientes distribuídos, CI/CD é obrigatório para garantir confiabilidade e velocidade.

O pipeline da Loja Veloz, implementado com GitHub Actions, realiza:

- checkout do código
- instalação de dependências
- lint e testes
- build multi-arch da imagem Docker
- push para o Docker Hub
- deploy automático via SSH
- uso seguro de secrets
- validações básicas

Essa automação garante que qualquer mudança no código seja refletida rapidamente em produção, sem intervenção manual.

6. Observabilidade: Logs, Métricas e Traces

Observabilidade é a capacidade de entender o comportamento interno de um sistema a partir de seus sinais externos.

6.1 Logs

Logs são essenciais para diagnosticar falhas. No projeto:

- logs da API são gerados via middleware
- logs dos containers podem ser acessados via Docker
- logs de produção seriam acessados via kubectl logs

6.2 Métricas

Métricas permitem monitorar tendências como:

- uso de CPU
- memória
- latência
- throughput

Para isso Prometheus e Grafana são boas escolhas porque formam o par mais consolidado de observabilidade em ambientes cloud-native: Prometheus coleta automaticamente as métricas necessárias para o HPA baseado em CPU já configurado no Kubernetes, enquanto Grafana transforma essas métricas em dashboards claros e acionáveis, permitindo acompanhar em tempo real o comportamento, a saúde e a escalabilidade da aplicação.

6.3 Traces

Para enxergar todo o caminho de uma requisição entre módulos da aplicação, identificando gargalos, falhas e dependências internas o uso de Traces seria indicado; por isso a arquitetura sugere OpenTelemetry para instrumentação e Jaeger ou Tempo para visualização dos spans, garantindo rastreabilidade distribuída mesmo que essa parte não tenha sido implementada no projeto, mas já esteja prevista e compatível com a estrutura atual.

7. Decisões Técnicas do Projeto

7.1 Containers

A decisão de utilizar containers foi motivada por:

- padronização do ambiente
- facilidade de deploy
- isolamento entre serviços
- compatibilidade com ARM
- redução de custos
- integração com CI/CD

7.2 Kubernetes

Mesmo não utilizado na prática, Kubernetes foi escolhido como:

- ambiente alvo de produção
- referência arquitetural
- base para escalabilidade futura
- padrão de mercado

Os manifests foram incluídos para demonstrar como o sistema subiria em um cluster real.

7.3 Pipeline CI/CD

O pipeline foi projetado para:

- automatizar build e deploy
- reduzir riscos

- garantir reprodutibilidade
- integrar com Docker Hub
- funcionar em ARM
- permitir evolução futura para Kubernetes

8. Estratégia de Deploy

A estratégia escolhida foi Rolling Update, pois:

- reduz downtime
- atualiza pods gradualmente
- permite rollback rápido
- é simples e eficiente para MVP

Alternativas consideradas:

- Blue/Green
- Canary

Ambas foram documentadas no README.

9. Estratégia de Escalabilidade

A escalabilidade planejada utiliza:

- Horizontal Pod Autoscaler (HPA)
- métricas de CPU
- possibilidade de métricas customizadas

Isso permite que o sistema escale automaticamente em períodos de pico.

10. Dificuldades Enfrentadas

Durante o desenvolvimento, enfrentei dificuldades reais que impactaram diretamente o projeto:

A primeira dificuldade foi para realizar os testes, primeiro com e depois Jest com TAP, por fim utilizei o padrao do Node Js, uma das dificuldade foi o uso do banco de dados para o teste que foi resolvido com um if na rota de produtos

Outra grande barreira foi a arquitetura ARM da VM gratuita da Oracle. Muitas imagens Docker não eram compatíveis, exigindo a criação de imagens multi-arch e ajustes no pipeline. Isso gerou erros complexos, especialmente quando o container subia e morria instantaneamente na VM.

Outra dificuldade foi o Minikube. Ao tentar rodar Kubernetes localmente, o Minikube ocupou portas essenciais, especialmente a porta 3000, gerando conflitos com o Docker Desktop. Isso causou erros como “port already allocated” e “docker engine not running”, que exigiram investigação profunda.

Também enfrentei limitações financeiras: não era possível rodar um cluster Kubernetes gerenciado, o que me obrigou a adaptar o projeto para demonstrar a arquitetura sem implantá-la integralmente.

Por fim, integrar Stripe, SMTP e PostgreSQL em containers exigiu ajustes finos de variáveis de ambiente, seeds e migrações.

Apesar das dificuldades, o projeto evoluiu de forma consistente e todas as decisões foram fundamentadas tecnicamente.

11. Conclusão

O projeto da Loja Veloz representa uma jornada completa de modernização cloud-native, integrando microserviços, containerização, CI/CD, orquestração e observabilidade. Mesmo com limitações práticas, a arquitetura foi construída seguindo padrões modernos e alinhada às necessidades reais da empresa.

A fundamentação teórica guiou cada decisão técnica, garantindo que o ambiente seja padronizado, seguro, escalável e pronto para evoluir para Kubernetes quando necessário. As dificuldades enfrentadas contribuíram para um aprendizado profundo e reforçaram a importância de automação, governança e telemetria em ambientes distribuídos.

12. Referências Bibliográficas

- Kubernetes Documentation — <https://kubernetes.io/docs>
- Docker Documentation — <https://docs.docker.com>
- 12-Factor App — <https://12factor.net>
- GitHub Actions Documentation — <https://docs.github.com/actions>
- Prometheus Documentation — <https://prometheus.io/docs>
- Grafana Documentation — <https://grafana.com/docs>
- OpenTelemetry — <https://opentelemetry.io/docs>
- Stripe API Reference — <https://stripe.com/docs/api>
- PostgreSQL Documentation — <https://www.postgresql.org/docs>
- CNCF Cloud Native Glossary — <https://glossary.cncf.io>